

Design an Efficient DNN Inference Framework with PS-PL Synergies in FPGA for Edge Computing

Wancheng Liu¹, Yuhai Li¹, Yining Yang¹, Jinyu Zhu², Longjun Liu^{2*}

¹Science and Technology on Electro-Optical Information Security Control Laboratory, Tianjin, China

²Institute of Artificial Intelligence and Robotics (IAIR), Xi'an Jiaotong University, Xi'an, China

*liulongjun@xjtu.edu.cn

Abstract—in recent years, Deep Neural Network (DNN) based methods have achieved great success for changing inflexible machine to intelligent and live system. Internet of Things (IoT) applications equipped with DNN in domains such as self-driving, speech processing and video surveillance particular challenges. Since DNNs are memory-/computing-intensive applications, FPGAs are customized to accelerate DNN on edge system due to their low latency and high energy efficiency. However, embedded CPU is usually essential for IoT system and single accelerator of FPGA is hard to gain maximized performance of the whole IoT system. Therefore, we design an efficient DNN inference framework with programming system (PS, e.g., ARM CPU) and programming logic (PL, e.g. FPGA) synergies for intelligent IoT system. Compared with the common IoT system only separately leverages FPGA for DNN accelerating and ARM for other tasks, our framework can make ARM and FPGA jointly accelerate DNN inference from the beginning. Our PS-PL co-design approach can take full advantage of the merits of ARM (control operations) and FPGA (parallel computing operations) for various DNN model accelerating. We implemented our co-design framework on Zynq SoCs. The experimental results show that our framework can achieve maximal 205.9 GOPs/s DNN inference accelerating under 6.2W while it would not affect other CPU tasks running on IoT system.

Keywords—DNN Accelerator, Edge-computing, ZYNQ, PS-PL Co-design, IoT System

I. INTRODUCTION

Over the past few decades, convolution neural networks (CNNs) have advanced at a breathless pace and have been gradually integrating into our lives (e.g., ubiquitous IoT devices) including image classification [1], speech recognition [2], and object recognition [3]. However, the structure of DNN models become increasingly complex and the training data shows explosive growth [4]. When classifying 1000 image categories of ImageNet, AlexNet with 5 layers and VGG-16 with 13 layers have about 60.95M and 138.34M parameters respectively.

The overwhelming numbers of DNN model parameters and various model structure causes tremendous challenges for the implementation of DNN inference computing in resource limited IoT system such as mobile platform and edge computing. Therefore, we use special DNN model accelerating hardware in general IoT systems for special DNN model accelerating.

The typical hardware platforms like low-power CPU, GPU and FPGA have attracted extensive interest for IoT system. Embedded FPGAs are one of the most attractive candidates to enable machine learning capability for IoT applications [5] because of their high energy efficiency and low cost. Therefore, FPGA based DNN model accelerators [6-9] have sprung up in recent years.

However, IoT system usually have embedded CPU system (e.g., ARM), only enhancing the IoT intelligence with customized FPGA accelerator without considering the CPU resources, which may lead to the CPU resources have not been fully utilized but single accelerator of FPGA is hard to gain maximized performance according to the whole IoT system.

Therefore, in this paper, we design an efficient DNN inference framework with PS-PL synergies for intelligent IoT system. We propose a multi-channel DNN parallel hardware accelerator based on Zynq SoCs. This approach allows CPU system to focus on DNN model analysis and control, and FPGA to focus on the execution of parallel computing.

Our contribution can be summarized as follow:

- We propose a DNN Inference Framework with PS-PL Synergies for intelligent IoT system, The PS side receives various DNN models, analyses parameters and sends the convolutional parameters to the PL side. The part of PL calculates each layer of DNN model according to the command from PS part.
- We present how to efficiently distribute the computing resource to the side of PS and the side of PL according their characteristic, how to share memory resources and how to exchange data between PS and PL.
- We implemented our framework on Xilinx Zynq SoC. The PL side is implemented with Verilog HDL code for calculating convolution, full connection, pooling and other layers. The PS side leverages different ARM Cortex A9 core to control the data flow of DNN model acceleration and other tasks for IoT system.

II. METHODS

Our framework is designed based on XILINX ZYNQ 7000 SoC architecture, in which ARM processor (PS) is utilized to perform data processing and model parameter analysis, while FPGA (PL) completes the calculation

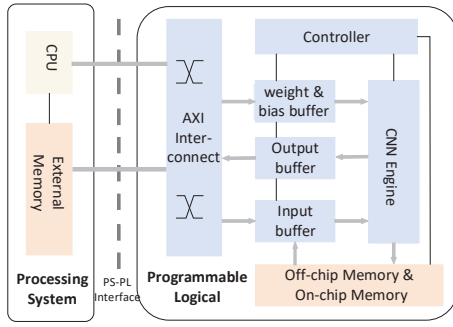


Fig. 1 The overview of .PS-PL Synergies for IoT intelligent system

according to the configuration parameters and data parameters sent by PS. The overview of the framework is shown in Figure 1. According to the characteristics of transmitted data between PS and PL side, three AXI HP interfaces (four ports are available) and one AXI GP interface are used in our framework. The GP interface mainly transmits the configuration parameters of the algorithm, and the HP interface mainly transmits the data parameters of the algorithm. The detailed design modules will be presented in the following subsection:

A. Convolution Unit

In the design of convolution layers, we must take the versatility and efficiency of convolution computing into account. The parallel architecture designed of this paper is based on input and output channel parallelism. The input feature maps and multiple three-dimensional weight parameters are obtained from the DDR memory, and the output feature maps of multiple channels are obtained concurrently according to the calculation rules of convolution. Our framework can flexibly configure the input and output channel parallelism as 4 vs 8, 8 vs 16 or 16 vs 32, according to the available hardware resources.

The specific implementation process is shown in Figure 2, which mainly includes the acquisition of convolution data and the calculation process. First of all, before the calculation, we will load the weight parameters from the PS side, store them into the on-chip BRAM and parse the cached data. Secondly, the input images are sent to PL side as a streaming. The time for the weight parsing and image transmission from PS to PL is overlapping, which will not cause additional delay. In order to support the general 3×3 and 5×5 convolutions, we use 5 FIFOs as sliding windows, which can not only improve the reusability of data, but also achieve padding operation without changing the streaming structure of the data. When the size of the convolution kernel is set to 3, we need to perform a zero-padded operation on the weight data, i.e., fill the corresponding invalid data area with 0, which will cause a waste of 16 multiplier resources (Hereafter referred to as non-critical multiplier). To this end, we perform three optimization schemes during calculations: (1) For non-critical multipliers, use LUT resources instead of DSP resources to build multipliers, because DSP is more precious than LUT computing resources (2) Skip the multiplication calculation when the input value is 0 (3) When the convolution kernel size is set to 3, turn off the clock of the multiplier in the non-critical area, reducing system power consumption.

Finally, according to the calculation law of convolution, the multiplication and accumulation operation of the

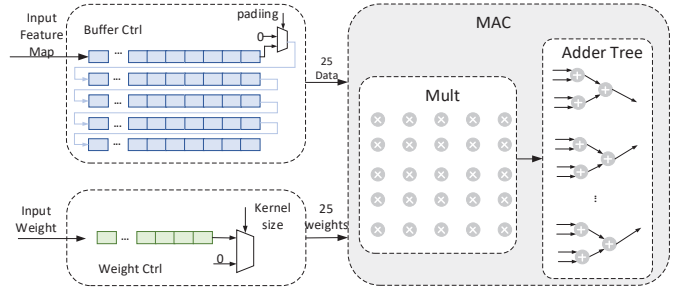


Fig.2. The overview of convolution unit

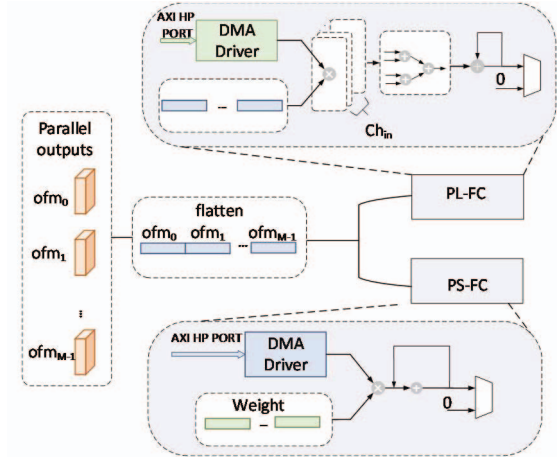


Fig. 3. The design of fully connected layer with different computing style

corresponding position of different channels is performed to complete the whole three-dimensional convolution calculation.

B. Fully connected layer (FC layer) Unit

The parameters of the FC layer sometimes are account for more than 90% of the whole DNN network. Therefore, we must focus on the benefits of data transmission overhead and the acceleration effect in PL side. Since the size of the implemented DNN network is different, it is difficult for us to set a unified standard to measure whether it is economical to perform the calculation of the FC layer on the PL side. Therefore, we have implemented the function of the fully connected layer on both the PL and PS side respectively, as shown in Figure 3. The PS side decides whether the calculation of the FC layer is performed on the PS side or the PL side, according to the structure of DNN model.

To implement FC layer on the PL side, we need to obtain all the weight parameters from the PS side through the AXI4 bus. The data of the feature map is obtained from the BRAM in PL side. As for the input channel parallelism of the entire network structure is less than the output channel parallelism, we need to rearrange the parameters data before calculating the FC layer. The time overhead can be saved by data pipeline, once the weight data sent by the PS side are received by PL side, the data will be repeatedly loaded from BRAM to perform the full connection calculation.

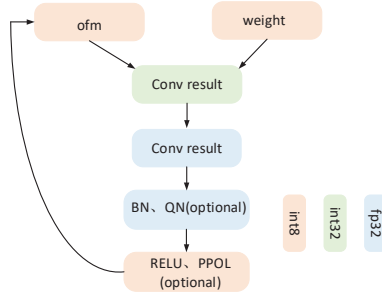


Fig. 4 The overview of integrated unit with the variation of bit width.

To implement the fully connected layer on the PS side, we only need to send the reorganized feature map data to the PS side. And the PS directly interacts with the DDR, i.e., reads the weight value from the memory for FC layer's calculation.

C. Integrated Unit

In our framework, we integrate quantization, BN, pooling and activation layer into one unit. Each module is optional and depends on the configuration parameters of DNN model in PS side, which is shown in Figure 4 (next page).

Among them, quantification is a very critical module because FPGA is not good at floating-point operations. The parameter value of the BN layer is relatively small, and the un-appropriate quantization for BN layer will cause a greater loss of accuracy for DNN network. Figure 4 shows the integrated unit in our design with variation of bit width. Weight parameters quantification is processed in PS side and sent to the PL side. The quantization of the intermediate result is performed on the PL side. Each layer of the PS side sends a scaling factor, and the PL side maps the intermediate result to an 8-bit interval, and then converts the floating-point number to a fixed-point number. In order to reduce resources, we perform fixed-point quantization of data before performing Relu and Pooling operation.

Although quantization has brought a slight decrease in the accuracy of the algorithm, the improvement in hardware performance is significant. First, the use of low bit width can utilize fewer computing resources and fewer internal storage resources. Second, the size of data access when the FPGA interacts with external DDR can be reduced.

D. Storage Unit

The data interaction strategy between PS and PL side largely determines the upper limit of DNN model acceleration. The data need to be processed by our framework includes three categories: (1) Model parameters of DNN model, including weights, offsets and initial images (2) Intermediate results of calculations (3) Input and output feature maps. Therefore, we leverage three kind of storage units: DDR3 in PS-side, on-chip BRAM in PL side and DDR3 in PL-side.

Due to the limited on-chip storage resources, we cannot completely map the intermediate results of the calculation process to on-chip resources. Therefore, our framework can flexibly configure all data to store in BRAM, DDR in PL-side and DDR in PS-side according to the structure of DNN model. In the current design, we store the initial model data into DDR3 on the PS side, and send the model parse data to the PL side through the AXI HP interface. The intermediate

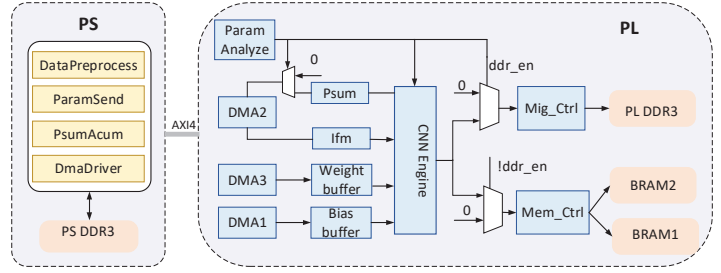


Fig. 5. The overview of storage distribution of our framework.

result of the network is stored into the DDR3 on the PS side or the on-chip BRAM on the PL side, and the convolution result is stored into the DDR3 or on-chip BRAM on the PL side.

The amount of intermediate data in each layer of the DNN model also determines how storage units are allocated. In our design, we set a soft threshold for data storage. When the calculation depth of the intermediate result is greater than this threshold, we leverage the released DMA resources to transmit the intermediate result data to PS side, and the result is accumulated on the PS side. After calculating the convolution result once, the results are sent to the PL side for the next layer of network. As the network data width is gradually becomes smaller until the data storage requirement is lower than the soft threshold, we put the intermediate results data into on-chip BRAM on the PL side instead of DDR in PS side. And perform accumulation operations for high access efficiency with no additional time overhead.

As our framework is PS-PL synergistic, the PS side can obtain the detailed information of current DNN model for accelerating and determines whether the data are stored on the off-chip DDR in PS side or off-chip / on-chip memory in PL side. The overall design of the storage is shown in Figure 5. In this design, the DDR3 storage space is converted into a continuous storage block space, and read or write operations are directed to different memory blocks. But for each read and write operation, the address space accessed is continuous, and bank switching will not be performed, resulting in a significant reduction in read and write efficiency.

III. EXPERIMENT

To evaluate the performance of our framework and the feasibility for DNN model acceleration, we deploy VGG and Mobilenet DNN model on Xilinx Zynq-7045 SoC. The PS works at 677MHz, while the PL works at 100MHz in the low-speed domain and at 150MHz in the high-speed domain. The computing performance of this architecture can be measured by GOPs/s. The equation for common calculation is as follow:

$$GOPs / s = (2 \cdot H \cdot W \cdot N \cdot M \cdot K \cdot K) / t \quad (1)$$

Where N/M is the number of input/output feature maps, H/W is the height/width of output feature map, K is the size of convolution kernel, t is the actual running time of that layer. There is a slight change in the calculation equation of depth separable convolution, e.g., Mobilenet:

$$GOPs / s = 2(H \cdot W \cdot M \cdot K \cdot K + H \cdot W \cdot M \cdot N) / t \quad (2)$$

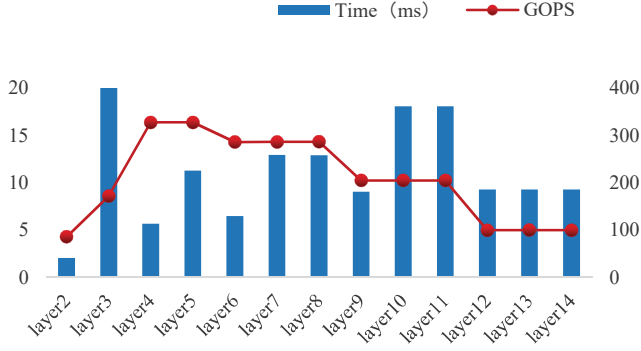


Fig. 6 The overview of storage distribution of our framework.

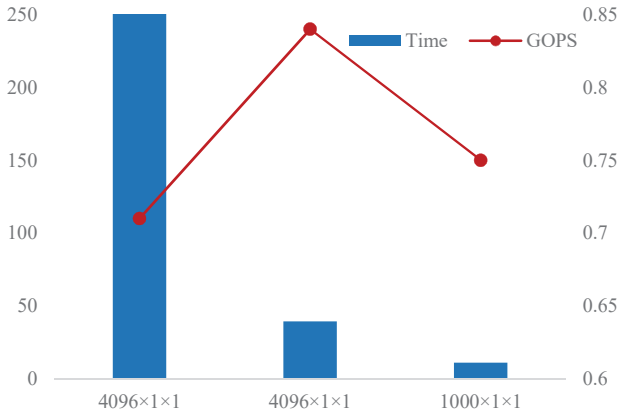


Fig. 7 The overview of storage distribution of our framework.

In CONV layer, the number of used DSP can be evaluated by the followed equation:

$$DSP_{num} = Para_{in} \times Para_{out} \times K_{size} \times K_{size} \quad (3)$$

For a typical design, $Para_{in}$ is normally set to 4, considering the size of the initial image and the characteristics of the hardware, and the output parallelism is limited to the memory and computation resource of the board. We adopt 16 MAC modules to parallel computing. The output feature maps calculated by CNN engine are stored into PL side memory, including DDR3 and BRAM.

A. The performance evaluation on VGG-16

The structure of VGG [10] [11] consists 15 computational blocks followed by three fully-connected layers. When testing execution time, we failed to take the data processing time into consideration. PS timer will start after we send the layer configuration parameter and end after receiving the interrupt signal from the PL. Figure 6 shows measured execution time and GOPS/s of 13 CONV layers accordingly. Evidently, this accelerator fits well with convolution structure, with a peak performance of 327 GOPS/s. It's not difficult to find that the performance of the first layer and last few layers are not as good as the middle one. As for the first layer, it is dominated by the bandwidth of AXI4 bus. Similarly, the widening of the network's depth inevitably causes additional interaction costs.

This is the reason why we prefer to put the calculation results on the PL-side rather than PS to reduce the data transmission

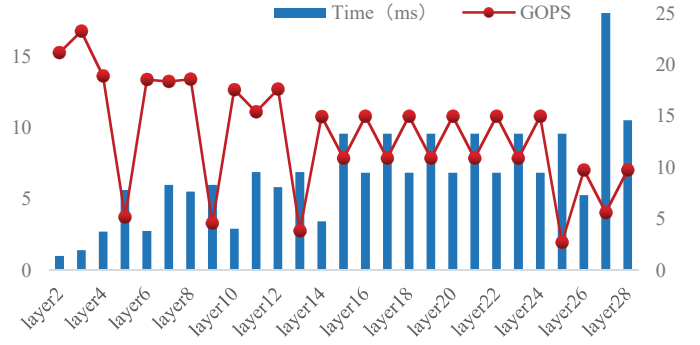


Fig. 8. The overview of storage distribution of our framework.

overhead. Instead, the FC layers don't get a good acceleration effect, detailed data can be seen in Figure 7. There are many reasons for this condition. Firstly, the FC layers are non-intensive computation task, which means we can not parallel FC layer very well. Secondly, the parameters of FC layer are enormous and stored in PS-side, which means the performance is highly dominated by the transfer bandwidth.

B. The performance evaluation on MOBILENET-V1

Depth separable convolution has been introduced into network to reduce the computational complexity of CONV layer. In this case, we don't implement the operation of average polling of 7×7 , so the final few layers are calculated by PS-side. It's interesting to note that the peak performance and average performance of mobilnetv1 is 23.28 GOPS/s and 13.18 GOPs/s, which can be seen in Figure 8. As may be noticed, the performance is far less than VGG-16. This is because the network sets the stride to be 2 for down sampling, but this does not save time and computing resources. What's more, 1×1 convolutions are also causing an idle state of many DSP resources, which will lead to the waste of computing resources.

TABLE I. PERFORMANCE COMPARISON

	Our work	Meloni et al.	Qiu et al.	Venieris et al.
Platform	Zynq Z-7045	Zynq Z-7045	Zynq Z-7045	Zynq Z-7045
Clock	150MHz	140MHz	150MHz	125MHz
Power (W)	6.24	~10	9.63	~10
DSP	787	864	780	900
LUT	154K	100K	183K	—
FF	128K	61K	128K	—
BRAM	537	320	486	—
Avg. GOPS/s	206(VGG16) 13(mobilenet-v1)	169 (VGG16) 58(resnet18)	138(VGG16) —	123 (VGG16) —
Frame/s	6.8 (VGG16) 5.5(mobilenet-v1)	5.5(VGG16) 6.6(resnet18)	4.46(VGG16) —	4.0 (VGG16) —
End-2-End	yes	yes	yes	yes
Quantization	8bit fixed	16bit fixed	16/8/4bit fixed	16bit fixed

C. Comparison with other works

A comparison with related works is presented in Table I. The architecture proposed by Meloni et al. [12], Qiu et al. [13] and Venieris et al. [14] claims a calculation performance up to 169 GOPs/s, 138 GOPs/s and 123 GOPs/s. In the case of Venieris et al., performance may be limited by the clock frequency and computation resource, for they have used all the DSP resources available by Zynq Z-7045. As for Qiu et al., the power of their work 35% higher than ours, while the performance is 33% lower. Meloni et al. is the strongest contender to our work, as their GOPs/s is 18% less than us. In comparison, they pay more attention to computing resources, while we prefer to concern more about storage resources. From the strict point of view, there is little sense in caring about resources between their work and ours, for they don't implement FC layer and SOFTMAX layer in PL-side. Although more resources consumed, we provide more choices for the processing system.

IV. CONCLUSION

The world has seen only rapid adoption of FPGAs for DNN acceleration. However, Internet of Things (IoT) system usually include embedded CPU processors. Therefore, this paper leverages PS (ARM) and PL (FPGA) co-design framework to accelerate DNN model on IoT platform to enhance the intelligence of IoT applications. We propose an efficient DNN inference framework with PS-PL synergies for intelligent IoT system. Our framework can make ARM and FPGA jointly accelerate DNN inference from the beginning. Our PS-PL co-design approach can take full advantage of the merits of ARM for control operations and FPGA for parallel computing operations for various DNN model accelerating. The experiments show that our framework can achieve 205.9 GOPs/s DNN accelerating under 6.2W while it would not affect other CPU tasks running on IoT system.

ACKNOWLEDGEMENT

This work was supported in part by National Defense Basic Scientific Research program of China under Grant JCKY202210C005, in part by the Natural Science Foundation of Shaanxi Province under Grant 2022JM-366, and in part by the China Postdoctoral Science Foundation under Grant 227556, and Grant 2020T130514.

REFERENCES

- [1] Seo, Yian , and K. S. Shin . "Image classification of fine-grained fashion image based on style using pre-trained convolutional neural network." 2018 IEEE 3rd International Conference on Big Data Analysis (ICBDA) IEEE, 2018.
- [2] Kurata, Gakuto . "Training deep neural network for acoustic modeling in speech recognition." (2017).
- [3] Shengna, W. U. , K. Wang , and Y. Ouyang . "Study on Small Samples SAR Image Recognition Detection Method Based on Transfer CNN." 2019 3rd International Conference on Electronic Information Technology and Computer Engineering (EITCE) 2019.
- [4] Sze, Vivienne , et al. "Hardware for machine learning: Challenges and opportunities." 2017 IEEE Custom Integrated Circuits Conference (CICC) IEEE, 2017.
- [5] Xiaofan Zhang et al. Machine learning on FPGAs to face the IoT revolution. In ICCAD, 2017.
- [6] T. Chen, Z. Du, N. Sun, J. Wang, and C. Wu, "DianNao: a smallfootprint high-throughput accelerator for ubiquitous machine-learning," in Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 269–284, 2014.
- [7] Temam, Olivier, "A defect-tolerant accelerator for emerging high-performance applications," in Proceeding of International Symposium on Computer Architecture (ISCA), pp. 356-367, 2012.
- [8] Q. Xiao, Y. Liang, L. Lu, S. Yan, Y. Tai, "Exploring Heterogeneous Algorithms for Accelerating Deep Convolutional Neural Networks on FPGA, in Proceedings of the Design Automation Conference (DAC) 2017.
- [9] X. Xu, D. Zeng, W. Xu, Y. Shi, an Y. Hu. "An Efficient Memristor-based Distance Accelerator for Time Series Data Mining on Data Centers. in Proceedings of the Design Automation Conference (DAC) 2017.
- [10] Simonyan, Karen , and A. Zisserman . "Very Deep Convolutional Networks for Large-Scale Image Recognition." Computer ence (2014).
- [11] Russakovsky, Olga , et al. "ImageNet Large Scale Visual Recognition Challenge." International Journal of Computer Vision (2015).
- [12] Meloni, Paolo , et al. "NEURAghe: Exploiting CPU-FPGA Synergies for Efficient and Flexible CNN Inference Acceleration on Zynq SoCs." Acm Transactions on Reconfigurable Technology & Systems (2017).
- [13] Qiu, Jiantao , et al. "Going Deeper with Embedded FPGA Platform for Convolutional Neural Network." the 2016 ACM/SIGDA International Symposium ACM, 2016.
- [14] Venieris, Stylianos I. , and C. S. Bouganis . "Latency-driven design for FPGA-based convolutional neural networks." International Conference on Field Programmable Logic & Applications IEEE, 2017.